

RAPPORT TECHNIQUE

Application Sona-Web

Démonstration de la Haute Disponibilité
sur Cluster Proxmox VE

Date : 12 mai 2026

Version : 2.0

Statut : Production Ready ✓

1. Présentation Générale

1.1 Qu'est-ce que Sona-Web ?

Sona-Web est une application web minimaliste conçue pour démontrer visuellement le bon fonctionnement de la haute disponibilité (HA) au sein d'un cluster Proxmox VE à trois nœuds physiques.

Objectif principal : migrer une VM applicative en live d'un nœud Proxmox vers un autre sans que l'utilisateur constate la moindre interruption de service.

L'application joue un rôle de témoin : toutes les requêtes HTTP sont enregistrées de manière asynchrone en base PostgreSQL, ce qui permet, après chaque migration, de consulter l'historique complet et de constater qu'aucune requête n'a été perdue.

1.2 Objectifs du projet

- Valider la migration live de machines virtuelles KVM sur stockage CEPH RBD.
- Mesurer le temps de bascule du load balancer HAProxy en cas de défaillance.
- Assurer la continuité de service grâce à la redondance applicative (deux instances Flask).
- Centraliser les données de monitoring dans une base PostgreSQL partagée.
- Offrir un dashboard de supervision en temps réel avec auto-refresh à 1 seconde.

2. Architecture Technique

2.1 Infrastructure physique

Composant	Rôle / IP
pve-node-1	Nœud Proxmox principal — 192.168.10.100
pve-node-2	Nœud Proxmox secondaire — 192.168.10.213
pve-node-3	Nœud Proxmox tertiaire — 192.168.10.84
PC personnel	Passerelle Internet + client — 192.168.10.36

2.2 Plan d'adressage des VMs

VM	Rôle	IP	Port	vCPU	RAM
VM-201	HAProxy (Load Balancer)	192.168.10.111	8080	1	1 Go

VM-202	Flask app-1	192.168.10.112	8000	1	1 Go
VM-203	Flask app-2	192.168.10.114	8000	1	1 Go
VM-204	PostgreSQL	192.168.10.115	5432	2	2 Go

2.3 Flux de données

Flux nominal (sans migration)

Utilisateur → HAProxy:8080 → Round Robin → app-1:8000 ou app-2:8000 → PostgreSQL (enregistrement asynchrone) → Dashboard (affichage temps réel).

Flux pendant une migration live

- Déclenchement de la migration de la VM app-1 depuis Proxmox.
- HAProxy détecte l'indisponibilité d'app-1 en moins de 200 ms (health check /health).
- Tout le trafic est redirigé automatiquement vers app-2.
- La VM app-1 redémarre sur le nouveau nœud Proxmox avec la même IP (statique).
- HAProxy détecte le retour d'app-1 en moins de 200 ms.
- Le trafic est à nouveau réparti équitablement entre les deux instances.

3. Composants Techniques et Justifications

3.1 Virtualisation : KVM (Debian 12)

Les machines virtuelles KVM sous Debian 12 ont été choisies pour leur isolation complète, leur compatibilité native avec la migration live Proxmox via CEPH RBD, et leur portabilité totale. Les conteneurs LXC ont été écartés en raison de contraintes de live migration plus complexes.

3.2 Stockage : CEPH RBD

Le stockage CEPH RBD (RADOS Block Device) est partagé entre tous les nœuds du cluster. Les disques des VMs étant accessibles depuis n'importe quel nœud, la migration live ne nécessite aucune copie de disque, ce qui réduit le temps d'indisponibilité à quelques millisecondes. Sans CEPH, Proxmox devrait copier l'intégralité du disque, entraînant plusieurs minutes d'interruption.

3.3 Load Balancer : HAProxy

HAProxy a été retenu car il est conçu exclusivement pour le load balancing et le proxying HTTP. Sa configuration offre une granularité très fine sur les health checks et les stratégies de bascule.

Paramètre HAProxy	Valeur et signification
-------------------	-------------------------

inter 200ms	Vérification de santé toutes les 200 ms
fall 1	1 seul échec suffit pour marquer un serveur DOWN
rise 1	1 seul succès suffit pour marquer un serveur UP
balance roundrobin	Répartition de charge équitable entre les instances
option redispatch	Réémission automatique vers une instance disponible

3.4 Application Web : Flask (Python)

Flask a été choisi pour sa simplicité de déploiement et sa légèreté, permettant de se concentrer sur la validation de la haute disponibilité sans complexité applicative superflue.

Fonctionnalités exposées :

- /health — endpoint dédié aux health checks HAProxy (retourne 'ok' si la DB est accessible).
- /api/status — état de l'instance et de la connexion PostgreSQL.
- /api/db-stats — statistiques globales : total requêtes, répartition par instance, taille BDD.
- /api/last-requests — les 20 dernières requêtes enregistrées.
- / — dashboard HTML avec auto-refresh à 1 seconde.

L'enregistrement en base de données est réalisé via un thread démon asynchrone, garantissant un impact nul sur le temps de réponse HTTP.

3.5 Base de Données : PostgreSQL

PostgreSQL centralise l'ensemble des données de monitoring. Toutes les instances Flask écrivent dans la même base, ce qui garantit un historique complet et cohérent, quel que soit le scénario de migration. SQLite a été écarté car son stockage local à chaque VM entraînerait une désynchronisation des données lors des migrations.

Schéma de la table principale :

- id — identifiant unique auto-incrémenté (SERIAL PRIMARY KEY).
- instance_id — identifiant de l'instance Flask ayant traité la requête (app-1 ou app-2).
- path — chemin de la requête HTTP.
- request_time — horodatage automatique (TIMESTAMP DEFAULT CURRENT_TIMESTAMP).
- client_ip — adresse IP du client (extraite de l'en-tête X-Forwarded-For).
- user_agent — chaîne User-Agent du navigateur ou client HTTP.

3.6 Haute Disponibilité : Proxmox HA

Le groupe HA Proxmox assure le failover automatique des VMs en 30 à 60 secondes en cas de panne d'un nœud physique. Combiné à la migration live CEPH, ce mécanisme couvre à la fois les pannes matérielles imprévues et les maintenances planifiées.

3.7 Adressage IP

Les VMs utilisent des adresses IP statiques configurées directement dans `/etc/network/interfaces`. Cette approche garantit la stabilité des IP après chaque migration, sans aucune dépendance à un serveur DHCP. La configuration HAProxy reste donc inchangée quelle que soit la localisation des VMs dans le cluster.

4. Configuration des Services

4.1 HAProxy — VM-201 (192.168.10.111)

Le fichier `/etc/haproxy/haproxy.cfg` configure le frontend sur le port 8080 et distribue le trafic en round-robin vers les deux instances Flask. Le health check `GET /health` est exécuté toutes les 200 ms avec une tolérance d'un seul échec (fall 1) pour une bascule quasi-instantanée.

4.2 Application Flask — VM-202 et VM-203

L'application `/root/app.py` est déployée à l'identique sur `app-1` et `app-2`. La variable d'environnement `INSTANCE_ID` différencie les deux instances dans les enregistrements en base. Le service est géré par `systemd` (`/etc/systemd/system/flask-app.service`) avec redémarrage automatique (`Restart=always`).

4.3 PostgreSQL — VM-204 (192.168.10.115)

PostgreSQL 17 est configuré pour écouter sur toutes les interfaces (`listen_addresses = '*'`). L'accès est restreint au réseau `192.168.10.0/24` via `pg_hba.conf` avec authentification md5. La base `sonadb` appartient à l'utilisateur `sona`.

4.4 Dashboard de Monitoring

Le dashboard web accessible sur `http://192.168.10.111:8080` affiche en temps réel :

- L'instance active (`app-1` ou `app-2`) et l'état de la connexion PostgreSQL.
- Le nombre total de requêtes et les requêtes de la dernière heure.
- La distribution des requêtes par instance avec pourcentages et barre de progression.
- Les 20 dernières requêtes avec horodatage, IP client et chemin.
- Un auto-refresh automatique toutes les 1 seconde.

5. Résultats des Tests

L'ensemble des scénarios de test ont été exécutés avec succès. Le tableau ci-dessous récapitule les résultats obtenus :

Test	Résultat	Temps de coupure	Requêtes perdues	Statut
Migration live app-1	Succès	< 200 ms	0	✓
Migration live app-2	Succès	< 200 ms	0	✓
Panne simulée app-1	Bascule automatique	< 200 ms	0	✓
Panne simulée app-2	Bascule automatique	< 200 ms	0	✓
Dashboard temps réel	Auto-refresh 1s	—	—	✓

Conclusion des tests : aucune requête n'a été perdue lors des migrations live ni lors des pannes simulées. Le temps de coupure observé est systématiquement inférieur à 200 ms, ce qui est imperceptible pour l'utilisateur final.

6. Synthèse des Choix Techniques

Composant	Choix retenu et justification principale
Virtualisation	VMs KVM Debian 12 — isolation complète, live migration native avec CEPH
Stockage	CEPH RBD — disques partagés entre nœuds, migration sans copie de disque
Load Balancer	HAProxy — health check 200 ms, redispach, bascule quasi-instantanée
Application	Flask (Python) — légèreté, simplicité, focus sur la validation HA
Base de données	PostgreSQL — centralisation des données, historique cohérent
Haute Disponibilité	Proxmox HA — failover automatique sur panne physique (30-60s)
Adressage réseau	IP statiques — stabilité après migration, pas de dépendance DHCP

7. Conclusion

L'application Sona-Web démontre avec succès qu'une migration live sur cluster Proxmox VE peut être totalement transparente pour l'utilisateur final. Les quatre piliers technologiques qui rendent cela possible sont :

- Le stockage partagé CEPH RBD, qui supprime la nécessité de copier les disques lors d'une migration.
- Le load balancer HAProxy, qui détecte toute indisponibilité en moins de 200 ms et redirige immédiatement le trafic.
- La double instance Flask, qui assure la redondance applicative et garantit la continuité du service.
- La base PostgreSQL centralisée, qui consolide un historique complet et cohérent de toutes les requêtes.

Les tests réalisés confirment que 0 requête n'a été perdue lors des migrations, que ce soit en conditions normales (migration planifiée) ou lors de pannes simulées. L'objectif de haute disponibilité est pleinement atteint.

Ce projet constitue une base solide et reproductible pour tout environnement nécessitant une haute disponibilité transparente avec Proxmox VE, CEPH et HAProxy.